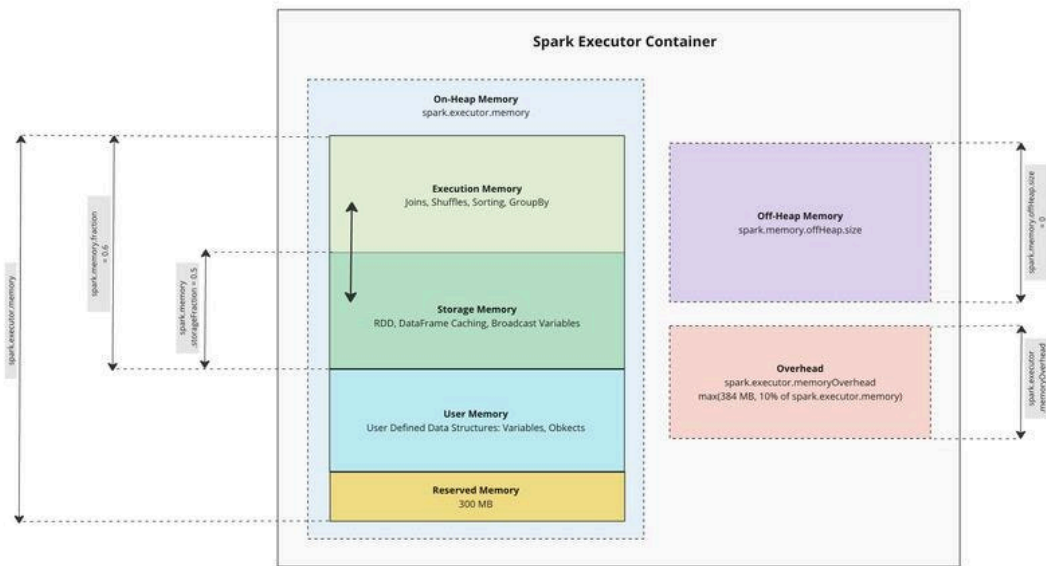


15 Process 100GB in Spark



1. Understand the Context First

If an interviewer asks, “How do you process 100GB of data in Spark?” Don’t jump into calculations right away. Instead, ask for more context. Even if they don’t provide additional details, it shows you understand that processing 100GB of data depends on multiple factors, including available resources.

Here are some key questions to consider:

1. What is the data format?

If the data is in CSV, converting it to Parquet can reduce its size to 10–20% of the original size, depending on the data, making processing more efficient.

2. What transformations are required?

Suppose we have a 100GB table with 1,000 columns, but we only need two columns and a filter like Age > 25. In that case, the data size will shrink significantly, reducing the needed resources.

3. Is the data coming from a single source or multiple sources?

This impacts how the data is read, partitioned, and processed.

4. What is the SLA (Service Level Agreement)?

Do we need to complete the job in 15 minutes or 150 minutes? The required speed affects how we optimize the job.

5. What’s the available cluster size?

Every organization has limited resources. You can’t just request 100GB of RAM for a single job. You need to work within constraints.

2. Define the Cluster Configuration

After the above discussion, the interviewer might provide details like:

(For example.) “You have 15GB RAM & 5 cores per node, with 5 nodes.”

If they don’t give specifics, you can make reasonable assumptions about cluster size and proceed accordingly. We can start our calculations with (15GB, 5 Cores) / Node, with 5 nodes. This is our starting point. Please understand, there’s no one right answer here. Based on our calculations and actual performance, we will redo these calculations, changing a few things. That’s what they call “Spark Executor Tuning” For now, let’s start our calculations with the following:

$15\text{GB} \times 5 = 75\text{GB}$,

$5 \text{ cores} \times 5 = 25 \text{ cores}$

Total raw cluster capacity: 75GB RAM, 25 cores

3. OS requirement (Hadoop/YARN/Kubernetes)

The rule of thumb is, we need to leave out 1GB of RAM and 1 core per node for the operating system for self-managed clusters using Hadoop/YARN. If you're using Databricks, you don't need to manually reserve memory and cores for OS daemons. Databricks handles this automatically, so you can skip this step.

Per node, we start with 15GB RAM and 5 cores. After reserving 1GB of RAM and 1 core per node, we have:

$5 \text{ cores} - 1 \text{ core} = 4 \text{ cores}$, $15\text{GB} - 1\text{GB} = 14\text{GB}$

For 5 nodes:

$14\text{GB} \times 5 = 70\text{GB}$, $4 \text{ cores} \times 5 = 20 \text{ cores}$

4. If you are using YARN (Optional)

If you're using YARN, the Application Master (AM) will need 1 core and 1GB of RAM from your cluster (often allocated as a separate container). If your cluster manager isn't YARN, you can skip this step.

From our totals of 70GB and 20 cores, we subtract:

$70\text{GB} - 1\text{GB for Application Master} = 69\text{GB}$

$20 \text{ cores} - 1 \text{ core} = 19 \text{ cores}$

5. How many executors?

We provide 3–5 cores per executor. If we give fewer than 3 cores to an executor, we have too many JVMs and high overhead. If we assign more than 5 cores, we get task contention and GC pauses.

No. of executors	Cores/executor	RAM/executor	Notes
3 executors	$19 \text{ cores} / 3 \text{ executors} = 6 \text{ cores / executor}$	$69\text{GB} / 3 \text{ executors} = 23\text{GB /executor}$ (1GB unused)	We only have 15GB & 5 cores/node
4 executors	$19 \text{ cores} / 4 \text{ executors} = 4 \text{ cores / executor}$ (3 cores unused)	$69\text{GB} / 4 \text{ executors} = 17\text{GB /executor}$ (1GB unused)	We only have 15GB ram per/node
5 executors	$19 \text{ cores} / 5 \text{ executors} = 3 \text{ cores / executor}$ (4 cores unused)	$69\text{GB} / 5 \text{ executors} = 13\text{GB /executor}$ (4GB unused)	Ideal configuration
6 executors	$19 \text{ cores} / 6 \text{ executors} = 3 \text{ cores/ executor}$ (1 cores unused)	$69\text{GB} / 6 \text{ executors} = 11\text{GB /executor}$ (3GB unused)	Can't accommodate 6 11GB executors in 5 nodes

Number of executors = 5

Memory/executor = 13GB (This will change in the next step)

Cores/executor = 3 cores

6. Calculating memory overhead (spark.executor.memoryOverhead)

Spark' executor memory = On-heap + overhead + off-heap (if enabled) memory

When we ask Spark for — **executor-memory** or **spark.executor.memory**, we are mentioning on-heap memory. Spark will add overhead and ask for resources to the cluster manager.

Total memory per executor = on-heap memory + Overhead + off-heap (Let's ignore this)

memory overhead = MAX(384MB, 10% of on-heap memory)

Across 5 executors: $5 \times (\text{on-heap} + \text{Overhead}) \leq 69\text{GB}$

$5(\text{on-heap}) + 5(\text{overhead memory}) = 69\text{GB}$

$5 (\text{on-heap}) + 5(\text{MAX}(384\text{MB}, 10\% \text{ of on-heap memory})) = 69\text{GB}$

10% of on-heap will be higher than 384 MB.

$$5(\text{on-heap}) + 5(0.1 * \text{on-heap}) = 69\text{GB}$$

$$5.5(\text{on-heap}) = 69\text{GB}$$

$$\text{On-heap memory} = 69\text{GB} / 5.5 \approx 12.54\text{ GB} \approx 12\text{ GB}$$

Now, we have to calculate overhead memory,

$$\text{memory overhead} = \text{MAX}(384\text{MB}, 10\% \text{ of on-heap memory})$$

$$\text{Memory overhead} = 0.1 * 12\text{GB} = 1.2\text{ GB}$$

$$\text{Total executor memory} = \text{on-heap memory} + \text{overhead memory} = 12 + 1.2 = 13.2\text{ GB}$$

Considering overhead memory, the 13GB (from the previous step) executor memory(on-heap) goes down to 12GB.

Executor size, excluding overhead memory: 12GB

Why can't we ask for 13GB of memory and add overhead memory on top of it?

$$(13\text{GB} * 10\% \text{ of } 13\text{GB} (1.3\text{GB})) = 14.3\text{ GB}$$

For 5 executors, $14.3\text{GB} * 5 = 71.5\text{GB}$, i.e. greater than the available 69 GB. So, we can't ask for 13GB of memory/executor.

Our final spark submit may look something like,

****spark-submit ****

- **master yarn \ # cluster manager**
- **deploy-mode cluster \ # Run in cluster mode**
- **num-executors 5 \ # 5 executors as requested**
- **executor-memory 12g \ # 12GB RAM per executor**
- **executor-cores 3 \ # 3 cores per executor**
- **driver-memory 4g \ # 4GB for driver**
- **driver-cores 2 \ # 2 cores for driver**

Big_job.py

7. Understanding partitioning and our configuration

By default, we have 128MB partitions in Spark.

Number of parallel tasks $\approx$ number of partitions $\approx$ number of cores

To process 100GB, we will need

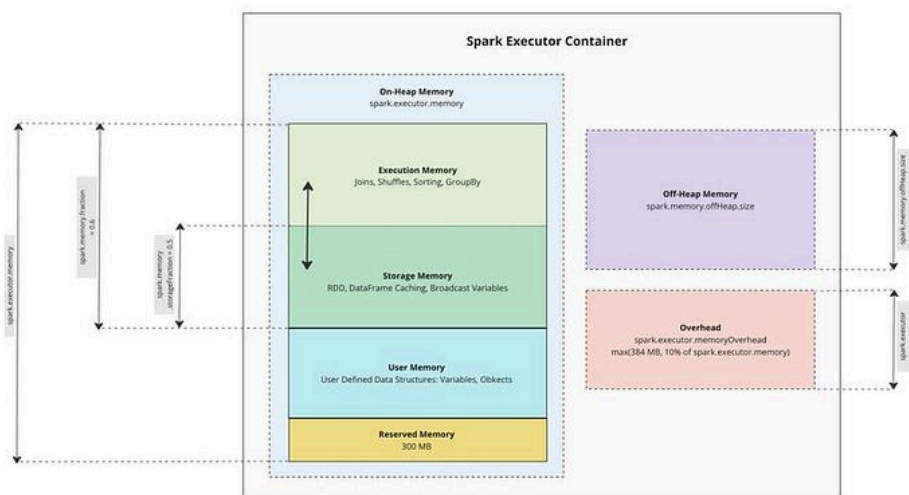
$$100000\text{MB} / 128\text{MB} \approx 781\text{ partitions.}$$

We have 3 cores per executor, totaling 15 parallel tasks.

$$781\text{ partition} / 15\text{ cores} \approx 52\text{ tasks of } 128\text{MB per core}$$

Spark will need to do 52 task iterations to process 100GB of data. If this doesn't suit our requirements, we can go back to step 2 and redo calculations.

8. Understanding Spark's Executor Memory



When we define **spark.executor.memory, 12g**, this executor memory has 4 parts.

Executor memory (on-heap memory) = Unified Memory (Storage memory + Execution memory) + User memory + Reserved memory

Storage Memory: Storing table data, caching datasets

Executor Memory: Actual execution memory

User Memory: Used for UDF, broadcast variable storage

Reserved Memory: Used for Spark's internal operations

Every executor needs 300MB of reserved memory.

$300\text{MB} \times 5 \text{ executors} = 1.5 \text{ GB}$ is reserved.

Effectively we have, $(12\text{GB} \times 5) - 1.5 \text{ GB} = 58.5\text{GB}$

spark.memory.fraction defines how much memory Unified memory (storage + execution) gets, the remaining will be assigned to user memory. Depending on the use case, we will tweak this. For example, we go ahead

with **spark.memory.fraction, 0.8**

$58.5\text{GB} \times 0.8\text{GB} = 46.8\text{GB}$ (Unified memory)

$58.5\text{GB} \times 0.2\text{GB} = 11.7\text{GB}$ (User memory)

spark.memory.storageFraction defines how much storage and execution memory is assigned in the unified memory. If we define **spark.memory.storageFraction 0.5**

50 % of 46.8GB (23.4GB) will be used for memory.

50% of 46.8GB (23.4GB) will be used for execution.

What if we don't need 23.4GB of memory for storage? We can tweak the settings! Note that the storage and execution memory partition is not fixed. (From Spark 1.6 and above) Spark can use some of the storage memory for execution if required or vice versa.

Heap per executor = 12GB

Reserved Memory = 300MB

Usable heap = 12GB — 0.3GB = 11.7GB

Unified Memory (0.8) = 9.36GB

User Memory (0.2) = 2.34GB

Total for 5 executors: 58.5GB + 1.5GB overhead

9. What if our requirements aren't met?

Let's assume we go ahead with the above calculations with 12 GB RAM, 3 cores per executor and 5 executors, and our SLA doesn't meet. We have a few options. We can redo calculations with increased memory. We can add one more executor. We can use cache/persist or not use it. We can tweak **spark.memory.fraction**. We can change **spark.memory.storageFraction**. We can also check for data skew, optimize joins (broadcast if small), and repartition data if tasks are unevenly distributed. We will need to do this based on our requirements. We will slowly get the (near) perfect configuration. I hope this answers, "How Do You Process 100GB of Data in Spark?"

10. Additional knowledge dump

We discussed almost all aspects of executor memory. We didn't discuss off-heap memory. I just want to write down some points here, so that there's nothing else left to discuss. Off-heap memory is not enabled by default. We will need to use **spark.memory.offHeap.enabled, true** to use off-heap memory. What's the size of off-heap memory? **spark.memory.offHeap.size** decides how much off-heap memory will be allocated. Usually, it should be 10–20% of the executor memory. Off-heap memory does not go through GC cycles like executor memory. Off-heap memory is divided into execution and storage memory, like unified memory in on-heap memory. Although it's a little slower than executor memory, it's better than spilling to disk.

THANK YOU FOR READING!